



Le stockage de structures de données dites « linéaires » est implémenté grâce à des listes, des piles ou des files.

Le stockage des structures hiérarchiques de données est implémenté grâce à des arbres.

On évoque dans ce chapitre des structures relationnelles de données.



Utilisées pour représenter une relation entre un ensemble d'objets homogènes :

✚ Réseaux routiers, de machines, sociaux, etc.

✚ Ordre d'exécution

Utilisées dans de nombreux algorithmes d'optimisation :

✚ Ordonnancement de tâches

✚ Routage réseau

✚ Jeux

Video Presentation des Graphes (16 min 20) :

https://www.youtube.com/watch?v=YYv2R1cCTa0&feature=emb_logo

(présentation générale : contextes et vocabulaire < 10 :00 - Connexité à 10 :00 – Arbre comme graphe particulier à 11 :00 – somme des degrés à 13 :10 – synthèse à 15 : 20)

I. STRUCTURES RELATIONNELLES

1. QUELQUES EXEMPLES

Ex 1 :

Parcours de Santé

On veut représenter un parcours de santé comportant 5 activités.

On peut passer de l'activité 1 aux activités 2 ,4, 5.

On peut passer de l'activité 2 aux activités 1, 3, 5

On peut passer de l'activité 3 aux activités 2, 4

On peut passer de l'activité 4 aux activités 1, 3

On peut passer de l'activité 5 aux activités 1, 2

Tom souhaite passer une seule fois par chaque activité.

un parcours possible pour Tom :

Zoë, elle, veut emprunter une fois et une seule fois chaque chemin.

Un parcours possible pour Zoë :

Exo 1. Réseau Social

Cinq réseaunautes, Ali, Bob, Camille, Donna et Emma sont connectés au même réseau social *instagrapp*.

A est ami avec tout le monde (mais curieusement pas avec lui-même)

B avec A et E

C avec A et D

D avec A et C

E avec A et B

1. Représenter les relations d'amitié à l'aide d'un graphe.

2. On peut représenter les relations entre les amis à l'aide d'un tableau.

	A	B	C	D	E
A	0	1	1	1	1
B					
C					
D					
E					

L'élément a_{XY} correspondant à la ligne de X et à la colonne de Y vaut 0 s'il n'y a pas de relation d'amitié entre X et Y , et 1 sinon.

Ce tableau A est appelé **matrice d'adjacence**. On note $A =$

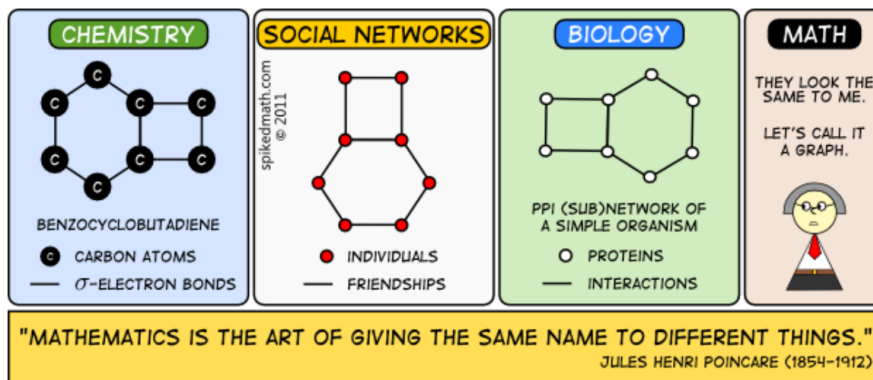
$$\begin{pmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{pmatrix}$$

3. Sur un autre réseau, 5 autres réseauteurs Urbain, Vicky, Xavier, Yann et Zoë sont connectés sur un autre réseau social, *whatsam*. La matrice d'adjacence est donnée par

$$W = \begin{pmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Tracer le graphe de ce réseau.

2. STRUCTURES DE GRAPHS



a) Sommets et arêtes

Un graphe non orienté est un ensemble de sommets (on parle aussi de nœuds) reliés par des arêtes.
 Une arête relie deux sommets qui sont alors voisins ou adjacents.
 Une arête est une boucle si elle relie un sommet à lui-même.

Exo 2. Dessiner tous les graphes non orientés ayant exactement trois sommets

Exo 3. Les logiciels de cartographie (ces logiciels sont souvent utilisés couplés à des récepteurs GPS)
 Soit les lieux suivants dans la commune de Grapheville : A, B, C, D, E, F et G.

1. Les différents lieux sont reliés par les routes suivantes :

- il existe une route entre A et C
- il existe une route entre A et B
- il existe une route entre A et D
- il existe une route entre B et F
- il existe une route entre B et E
- il existe une route entre B et G
- il existe une route entre D et G
- il existe une route entre E et F

représentation sous forme de graphe

représentation sous forme matricielle :

$$G = \begin{pmatrix} & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \end{pmatrix}$$

Les arêtes figurent une circulation à double sens : le graphe est non orienté.

2. Nouvelle municipalité, nouveau plan de circulation : les différents lieux sont maintenant reliés par les routes suivantes :

- il existe une route entre A et C (double sens)
- il existe une route entre A et B (sens unique $B \rightarrow A$)
- il existe une route entre A et D (sens unique $A \rightarrow D$)
- il existe une route entre B et F (sens unique $B \rightarrow F$)
- il existe une route entre B et E (sens unique $E \rightarrow B$)
- il existe une route entre B et G (double sens)
- il existe une route entre D et G (double sens)
- il existe une route entre E et F (double sens)

représentation sous forme de graphe

représentation sous forme matricielle :

$$G = \begin{pmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Un graphe peut être orienté (les arêtes portent une direction et on parle alors d'arcs plutôt que d'arêtes)

3. On précise les distances entre chaque lieu : elles apparaissent alors dans les coefficients de la matrice dans le cas d'une arête existante.

$$G = \begin{pmatrix} 0 & 3 & 2 & 4 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 4 & 3 & 6 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 5 \\ 0 & 4 & 0 & 0 & 0 & 7 & 0 \\ 0 & 3 & 0 & 0 & 7 & 0 & 0 \\ 0 & 6 & 0 & 5 & 0 & 0 & 0 \end{pmatrix}$$

représentation sous forme de graphe pondéré

Rq 1. Une autre pondération peut être réalisée avec le temps nécessaire pour relier deux sommets.

Une arête peut porter une valuation (un poids, un coût).

Exo 4.

1. Que peut-on dire de la matrice d'adjacence d'un graphe non orienté ?
2. Combien y a-t-il de graphes orientés ayant exactement 3 sommets ? on ne demande pas de les dessiner mais seulement de les dénombrer.

b) Ordre, degré, voisinage

L'ordre d'un graphe est le nombre de ses sommets ;

Le degré d'un sommet est le nombre d'arêtes issues de ce sommet ;

Le sommet t est un sommet adjacent au sommet s si il existe un arc allant de s à t .

Le voisinage d'un sommet s est l'ensemble des sommets adjacents (ou voisins) à s .

Rq 2. Que peut-on dire de la somme des degrés d'un graphe non orienté, sans boucle ?

Exo 5. Dans les réseaux *whatsam* et *instagrapp* précédents :

	<i>instagrapp</i>	<i>whatsam</i>
Ordre		
Degré de	A :	U :
Degré de	B :	X :
Voisinage de	E :	V :

Un chemin ou une chaîne est une suite d'arêtes consécutives dans un graphe. On la désigne par les lettres des sommets qu'elle comporte.

Un cycle est une chaîne qui commence et se termine au même sommet.

Exo 6. Modélisation d'un réseau routier

Les graphes sont abondamment utilisés par les logiciels de cartographie : Les sommets représentent les villes et les arêtes sont les routes qui les relient.

🚦 Certaines routes peuvent être à sens unique : on utilisera alors des arêtes orientées.

🚦 Les distances entre les villes peuvent être prises en compte sous forme de poids associé aux arêtes.

On considère les villes A, B, C, D, E, F et G. le réseau routier est organisé de la manière suivante :

- | | | |
|--|-----------|-----------|
| a. A et C sont reliées par une route à double sens ; | | AC = 2 km |
| b. B est reliée à A par une route à sens unique | (B → A) ; | BA = 3km |
| c. A est reliée à D par une route à sens unique | (A → D) ; | DA = 4km |
| d. B est reliée à F par une route à sens unique | (B → F) ; | BF = 3km |

- e. E est reliée à B par une route à sens unique (E → B) ; EB = 4km
- f. B et G sont reliées par une route à double sens ; BG = 6km
- g. D et G sont reliées par une route à double sens ; DG = 5km
- h. E et F sont reliées par une route à double sens ; EF = 7km

- Donner une représentation graphique de ce graphe.
- Ecrire la matrice d'adjacence de ce graphe. Les coefficients dans la matrice seront les distances entre les villes.

II. REPRÉSENTATION ET IMPLEMENTATION EN PYTHON



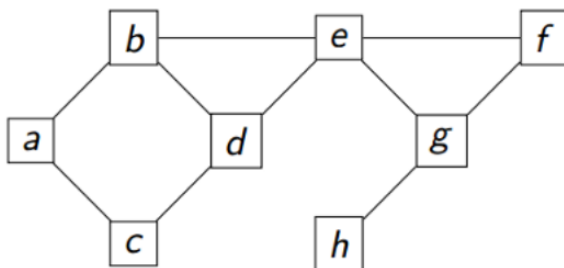
Il y a de nombreuses représentations et implémentations possibles. On en présente deux qui permettront la construction d'un graphe et le parcours de ses sommets ou de ses arcs.

1. MATRICE D'ADJACENCE

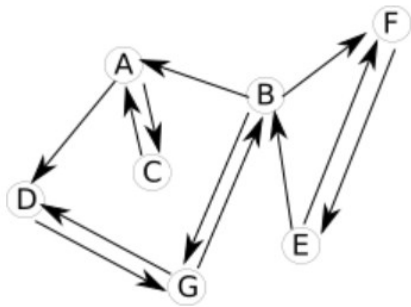
Les sommets du graphe sont supposés être des entiers entre 0 et $n - 1$, n étant le nombre total de sommets. On peut alors représenter le graphe en identifiant sa matrice à un tableau à deux dimensions de $n \times n$ booléens.

Exo 7. A partir du fichier `..\SRC\graphe_matrice_adjacence_tbc.py`:

- Compléter les méthodes de la classe Graphe dont on donne les *docstrings*.
- Construire une instance de cette classe représentant le graphe **g1_test** suivant dans lequel on renommera les sommets en entiers de 0 à 7 :



3. Construire une instance de cette classe représentant le graphe **g2_test** suivant dans lequel on renommera les sommets en entiers de 0 à 6 :



4. Ecrire et tester les fonctions suivantes à l'aide notamment des graphes **g1_test** et **g2_test** :
- fonction **nb_arcs(g)** qui renvoie le nombre d'arcs du graphe **g** passé en paramètre.
 - fonction **nb_arcs_db_sens(g)** qui renvoie le nombre d'arcs à double sens du graphe **g** passé en paramètre.
 - Fonction **degre(g,s)** qui renvoie le degré de **s** dans le graphe **g**.
 - fonction **sommet_degre_max(g)** qui renvoie le sommet de plus haut degré du graphe **g** passé en paramètre.
 - Fonction **connexion(g,s)** qui renvoie la liste des voisins de **s** ou ceux dont **s** est un voisin dans le graphe **g**.
5. Quelle est la taille occupée en mémoire un graphe de 1000 sommets représenté par sa matrice d'adjacence ?



[..\SRC\graphe_matrice_adjacence.py](#)

2. DICTIONNAIRE D'ADJACENCE



Dans cette nouvelle représentation, un graphe est représenté par un dictionnaire qui associe à chaque sommet (la clé) la liste de ses voisins (la valeur).

Exo 8. Tester les quelques rappels suivants sur les dictionnaires :

```

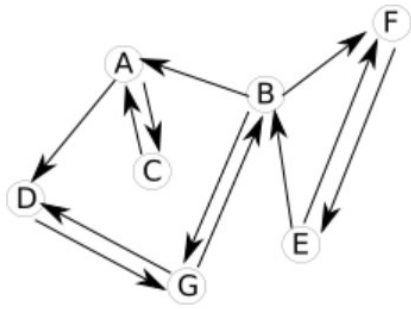
G={"A":-2,"B":3,"C":5}.
# G est un dictionnaire
print("affiche les clés",G.keys()) # affiche les clés
print("affiche les valeurs",G.values()) # affiche les valeurs
print("affiche les clés,valeurs dans une liste",list(G.items()))
print("affiche les clés dans une liste",list(G.keys()))
print("affiche les valeurs dans une liste",list(G.values()))

print(len(G)) # affiche le nombre de clés
print(G['e'])# affiche la valeur de la clé 'e'
# G.keys() et G.values() sont itérables
# affiche les valeurs du dictionnaire
for el in G.values():
    print(el)
#affiche les clés et les valeurs des clés
for key in G.keys():
    print(key,G[key])
for key in G :
    print(key,G[key])
  
```

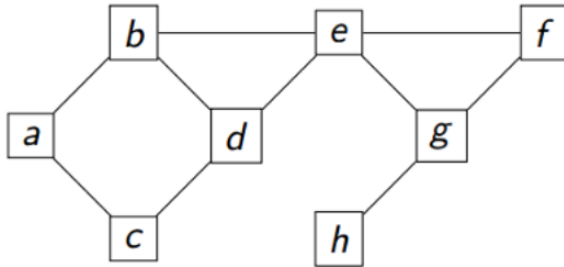
Exo 9. A partir du fichier [..\SRC\graphe_dictionnaire_adjacence_tbc.py](#):

1. Compléter les méthodes de la classe Graphe dont on donne les *docstrings*.

2. Construire une instance de cette classe représentant le graphe **g2_test** suivant :



3. Construire une instance de cette classe représentant le graphe **g1_test** suivant:



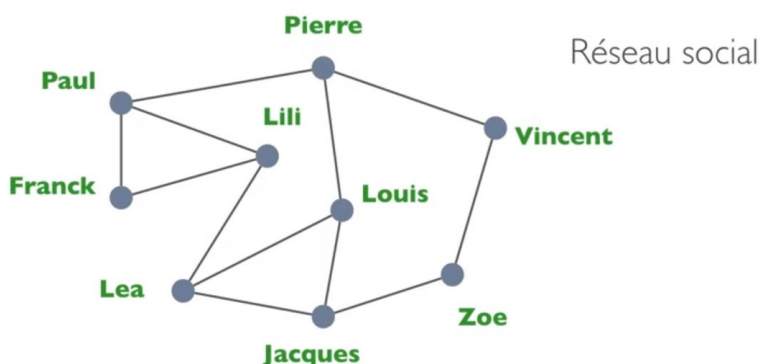
La représentation d'un graphe non orienté est conforme à celle d'un graphe orienté : elle se contente de doubler les arcs existant pour garantir l'invariant suivant :
Il existe un arc reliant s à t si et seulement si il existe un arc reliant t à s .

- Rq3.* La création d'une arête entre s et t se traduira par la création d'un arc de s à t et d'un arc de t à s .
4. Ecrire et tester les fonctions suivantes à l'aide notamment des graphes **g1_test** et **g2_test** :
- fonction **nb_arcs(g)** qui renvoie le nombre d'arcs du graphe g passé en paramètre.
 - fonction **nb_arcs_db_sens(g)** qui renvoie le nombre d'arcs à double sens du graphe g passé en paramètre.
 - Fonction **degre(g,s)** qui renvoie le degré de s dans le graphe g .
 - fonction **sommet_degre_max(g)** qui renvoie le sommet de plus haut degré du graphe g passé en paramètre.
 - Fonction **connexion(g,s)** qui renvoie la liste des voisins de s ou ceux dont s est un voisin dans le graphe g .
5. Matrice d'adjacence ou dictionnaire d'adjacence ?...
- Quels avantages présente la représentation par dictionnaire d'adjacence par rapport à la représentation par matrice d'adjacence ?
 - Quelles adaptations envisager pour cette représentation dans le cas d'un graphe pondéré ?



..\SRC\graphe_dictionnaire_adjacence.py

Exo 10. Implémenter le graphe du réseau social ci-dessous et faire afficher le nom du réseaunaute qui a le plus d'amis.

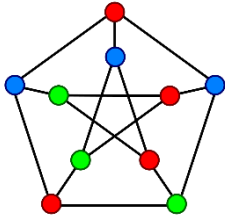


III. EXOS LONGS

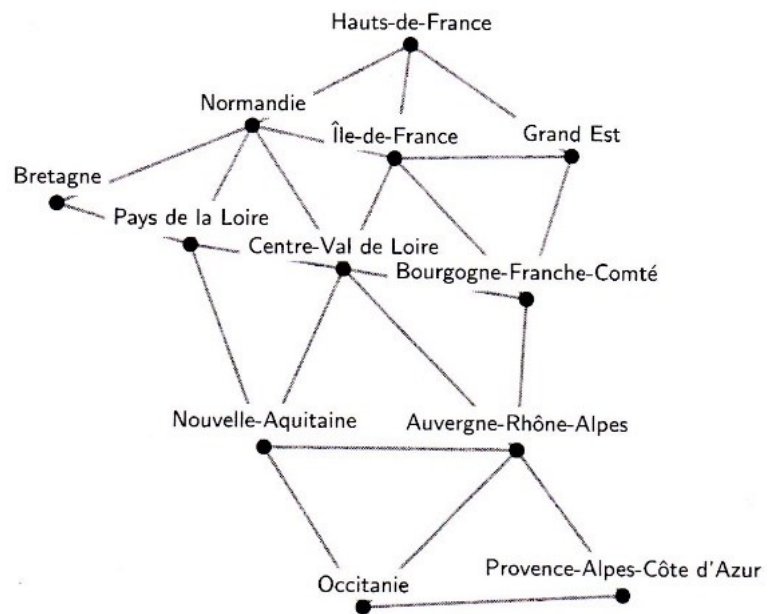
Exo 11. [TD Graphe Chifoumi.docx](#)

Exo 12. Coloration Gloutonne

On considère le graphe des régions de France métropolitaine. Les arêtes figurent deux régions limitrophes.



On cherche à colorier le graphe c'est-à-dire associer une couleur à chacun de ses sommets de façon à ce que deux sommets voisins n'aient pas la même couleur.



Dans le cas des régions françaises, la conséquence immédiate de la coloration du graphe est une carte qui peut ressembler à celles-ci-dessous.

1. La carte de droite est à colorier (sans dépasser) avec moins de 6 couleurs.



12 couleurs

Les 13 régions de la France métropolitaine (2016)



? couleurs

Colorier le graphe avec un nombre de couleurs minimal est un problème difficile. On peut cependant envisager un algorithme glouton qui fournira une solution même non optimale.

On rappelle le principe de l'algorithme glouton (cf le problème du sac-à-dos) :

On procède par étapes et, dans l'objectif de la meilleure solution possible, la stratégie (naturelle) de l'algorithme **glouton** consiste à choisir à chaque étape une option qui semble **momentanément optimale**. La question se pose ensuite de savoir si cette succession de choix momentanément optimaux, fournit, ou non, une solution finalement optimale.

Ici,

-  les couleurs sont représentées par des entiers naturels
-  La fonction **colorer(g)** prend le graphe en paramètre et renvoie le coloriage (qui est un dictionnaire associant à chaque sommet une couleur) et le nombre total de couleurs utilisées :
pour chaque sommet :
le choix glouton consiste à choisir la plus petite couleur non utilisée par les voisins
-  Le choix glouton est réalisé par la fonction **min_exclu(voisins,coloriage)** qui prend en paramètre la liste des voisins du sommet à colorer, le coloriage en construction, et renvoie la plus petite couleur non utilisée par les voisins.

Rq 4. Ce type d'algorithme ne remet jamais en cause une décision prise auparavant (pas de retour en arrière (*backtracking*) dans la construction de la solution)



[..\SRC\graphe_coloriage_glouton.py](#)

2. Implémenter le graphe des régions métropolitaines (en utilisant soit la matrice d'adjacence soit le dictionnaire d'adjacence, donc en important l'un des deux fichiers implémentant la classe **Graphe**)
3. Coder la fonction **valide(g, dico_loriage)** qui valide le dictionnaire de coloriage **dico_loriage** appliqué à un graphe **g**, c'est-à-dire qui renvoie **True** si tous les sommets sont coloriés avec une couleur différente de celle de ses voisins.
4. Coder les deux fonctions **colorer(g)** et **min_exclu(voisins,coloriage)**.
5. Combien de couleurs nécessite la coloration du graphe des régions par cet algorithme ? Est-ce la solution optimale ?



[SY Graphe Coloration.docx](#)